

Obsidian de l'intérieur

Architecture inférée du code d'Obsidian : patterns, flux de données, points d'extension et bibliothèques — en vue de la conception d'un clone sous Electron.

Préparé pour Julien — 6 juillet 2026

État des connaissances : Obsidian $\approx$ v1.12 (API `obsidian.d.ts` 1.12.x, typages communautaires 1.12.7)

Document de synthèse — Obsidian n'est pas open source ; tout le contenu est inféré de sources publiques.

Résumé. Obsidian est une application Electron sans framework UI, organisée autour d'un graphe d'objets unique (App) et de quatre piliers : un arbre de fichiers abstrait (Vault + DataAdapter), un index de métadonnées incrémental hors thread principal (MetadataCache, Web Worker + IndexedDB, adressage par hachage de contenu), un gestionnaire de fenêtrage en arbre composite sérialisable (Workspace), et un éditeur CodeMirror 6 masqué derrière une façade stable. Deux patterns transversaux structurent tout le code : Component (cycle de vie hiérarchique à nettoyage automatique) et Events (pub/sub typé à références désinscriptibles). Le système de plugins n'est pas une pièce rapportée : les fonctionnalités « cœur » sont elles-mêmes des plugins internes consommant la même API. Ce document reconstitue ces mécanismes, les hooks offerts aux développeurs, les flux critiques (démarrage, édition, indexation, renommage) et la liste exacte des bibliothèques tierces, puis en tire une feuille de route pour un clone.

Sommaire

1. Méthodologie : comment lire une architecture fermée	3
2. Vue d'ensemble : processus, bundle, plateformes	4
3. Le graphe d'objets App et le pattern « registres »	6
4. Component : le cycle de vie hiérarchique	8
5. Events : le pub/sub typé et la carte complète des hooks	9
6. Vault & DataAdapter : la persistance	12
7. MetadataCache : l'index incrémental	14
8. Workspace : fenêtrage composite et vues différées	16
9. L'éditeur : CodeMirror 6 sous une façade	18
10. Le rendu Markdown en mode lecture	20
11. Le système de plugins	21
12. Le toolkit UI maison	23
13. La vue graphe : PixiJS + simulation en worker	25
14. Recherche et correspondance floue	26
15. Réseau, secrets, sécurité	27
16. Les flux critiques, pas à pas	28
17. Bibliothèques externes : la liste exacte	30
18. Feuille de route pour votre clone	31
19. Annexe : hiérarchie de classes consolidée	33
Sources	34

1. Méthodologie : comment lire une architecture fermée

Obsidian n'est pas open source, mais c'est un cas presque idéal de rétro-conception *documentaire* : pour faire vivre son écosystème de plugins, l'éditeur publie volontairement une quantité inhabituelle d'informations structurelles. Quatre familles de sources ont été croisées pour ce document, chacune associée à un badge de confiance utilisé dans tout le texte.

Badge	Source	Fiabilité
API	Typages officiels <code>obsidian.d.ts</code> (paquet npm <code>obsidian</code> , 8 482 lignes, v1.12.x), documentation développeur <code>docs.obsidian.md</code> , CHANGELOG officiel de l'API.	Contractuelle. C'est la surface que l'éditeur s'engage à maintenir ; elle reflète directement les classes réelles (les typages sont « générés automatiquement » depuis le code source, comme l'indique leur en-tête).
CRED	Page officielle des crédits (<code>help.obsidian.md/credits</code>) : liste exhaustive des bibliothèques tierces embarquées, avec versions.	Officielle, mise à jour avec l'application.
COMM	Typages communautaires <code>obsidian-typings</code> (projet Fevol et contributeurs) : 1,47 Mo de déclarations reconstituées par inspection du bundle, versionnées par release d'Obsidian (ici 1.12.7). Ils décrivent les managers <i>internes</i> non exposés par l'API publique.	Très fiable factuellement (noms de champs et signatures observés dans l'app), mais non contractuelle : tout peut changer sans préavis.
INF	Inférences raisonnées de l'auteur : quand plusieurs indices convergent sans preuve directe.	À vérifier par vous-même ; le raisonnement est toujours donné.

Deux remarques de rigueur. D'abord, une API publique ne montre que ce qu'on veut bien montrer ; mais dans le cas d'Obsidian, l'API expose les *vraies* classes internes (elles ne sont pas des DTO de façade : `App`, `Vault`, `Workspace` sont les objets vivants de l'application, ce que confirme la possibilité pour les plugins de les monkey-patcher). L'architecture lue ici est donc l'architecture réelle, vue avec des zones d'ombre. Ensuite, un point juridique : la licence d'Obsidian interdit la décompilation de l'application ; ce document n'en fait pas et s'appuie exclusivement sur des publications volontaires (typages npm, documentation) et sur le travail public de la communauté. Pour votre clone, la conséquence pratique est la même que pour toute réimplémentation en salle blanche : s'inspirer des *concepts* est licite, copier le code, les assets, le nom ou l'apparence distinctive ne l'est pas. Je ne suis pas juriste ; pour un produit destiné à la distribution, faites valider ce point.

Le fil conducteur du document

À chaque section, la question n'est pas seulement « comment Obsidian fait-il ? » mais « quelle contrainte ce choix résout-il ? ». Un clone qui copie les formes sans les contraintes reproduit les coûts sans les bénéfices.

2. Vue d'ensemble : processus, bundle, plateformes

2.1 Un renderer qui contient toute l'application

Obsidian desktop tourne sur Electron 37 **CRED**. Chaque fenêtre principale correspond à un coffre (*vault*) : un dossier ordinaire du disque. La quasi-totalité de l'application vit dans le *processus renderer* : le graphe d'objets `App`, le système de fichiers virtuel, l'indexeur, l'éditeur, les plugins. Rien dans l'API plugin ne fait référence à des services du processus principal ; les plugins s'exécutent dans le renderer avec, sur desktop, l'intégration Node activée (ils peuvent faire `require("fs")` ou `require("electron")`) **API**. Le processus principal se réduit vraisemblablement au strict nécessaire : création de fenêtres, menu natif, enregistrement du protocole `obsidian://`, requêtes HTTP sans CORS pour `requestUrl()`, dialogues de fichiers et mise à jour automatique **INF**.

C'est un choix structurant : pas d'architecture multi-processus à la VS Code (extension host séparé, RPC). Obsidian privilégie la latence nulle et la simplicité d'un espace mémoire unique, et paie le prix en isolation (un plugin peut geler ou compromettre l'app). La section 11 montre que ce compromis est assumé et compensé par ailleurs (revue humaine, mode restreint, API de désinscription systématique).

2.2 Pas de framework UI

Ni React, ni Vue, ni Svelte : l'interface est en TypeScript impératif sur DOM nu, avec un micro-toolkit maison (section 12). Trois indices concordants : l'API publique manipule des `HTMLElement` partout ; les prototypes DOM sont étendus par des helpers de création d'éléments (`createElement`, `createDiv`...) qui n'auraient aucun sens sous un vDOM ; et l'équipe l'a confirmé publiquement à plusieurs reprises **INF**. Le bundle applicatif unique (`app.js`) est assemblé avec Webpack **CRED**.

2.3 Mobile : même cœur, autre coque

Les applications iOS/Android embarquent le même cœur web dans Capacitor 5 **CRED**. La portabilité repose sur trois mécanismes précis : l'accès disque passe par l'interface `DataAdapter` avec deux implémentations, `FileSystemAdapter` (Node) et `CapacitorAdapter` (ponts natifs) **API** ; l'objet global `Platform` expose des drapeaux (`isDesktopApp`, `isIosApp`, `isPhone`, `isTablet`...) **API** ; et l'UI mobile substitue des conteneurs dédiés dans le même arbre de fenêtrage (`WorkspaceMobileDrawer` au lieu des `sidedocks` **API**, plus `MobileNavbar`, `MobileToolbar`, `MobileTabSwitcher` **COMM**). Les plugins déclarent `isDesktopOnly` dans leur manifeste s'ils dépendent de Node **API**.

2.4 Multi-fenêtres : plusieurs documents, un seul monde JavaScript

Depuis la v0.15, on peut détacher des onglets en fenêtres flottantes (*popouts*). Le détail d'implémentation est élégant et entièrement lisible dans les typages : il n'y a qu'un **seul contexte JavaScript** pour toutes les fenêtres. Preuves **API** : les globaux `activeWindow` et `activeDocument` ; les propriétés `win`, `doc` et `constructorWin` ajoutées à Node ; le hook `HTMLElement.onWindowMigrated(cb)` appelé quand un élément change de fenêtre ; les événements `window-open`/`window-close` du `Workspace`. Les fenêtres secondaires sont ouvertes de manière à partager le contexte de la fenêtre mère (technique `window.open` d'Electron) **INF**, si bien que les plugins n'ont pas à être multi-instanciés : un seul état, plusieurs Document.

Conséquence pour un clone

Si vous prévoyez les popouts, décidez du modèle « un realm, N documents » dès le début : il contamine tous les helpers DOM (ne jamais capturer `document` global, toujours `el.doc`/`el.win`) et interdit les `instanceof`

HTMLélément naïfs entre fenêtres — d'où le helper `instanceOf()` qu'Obsidian ajoute à Node pour comparer par-delà les realms **API**.

2.5 La philosophie « files over app »

Le coffre est un dossier de fichiers Markdown ordinaires ; tout état dérivé (index, layout, configuration) est soit un petit fichier JSON dans `.obsidian/`, soit un cache jetable et reconstituable (IndexedDB). Cette décision irrigue l'architecture entière : l'indexeur doit tolérer des modifications externes concurrentes (section 6.4), la configuration est faite de fichiers plats surveillés et sauvegardés avec debounce (section 6.6), et la synchronisation peut être un simple service de fichiers chiffrés (section 15). C'est la contrainte n°1 à conserver dans un clone : elle est le fondement de la confiance des utilisateurs d'Obsidian.

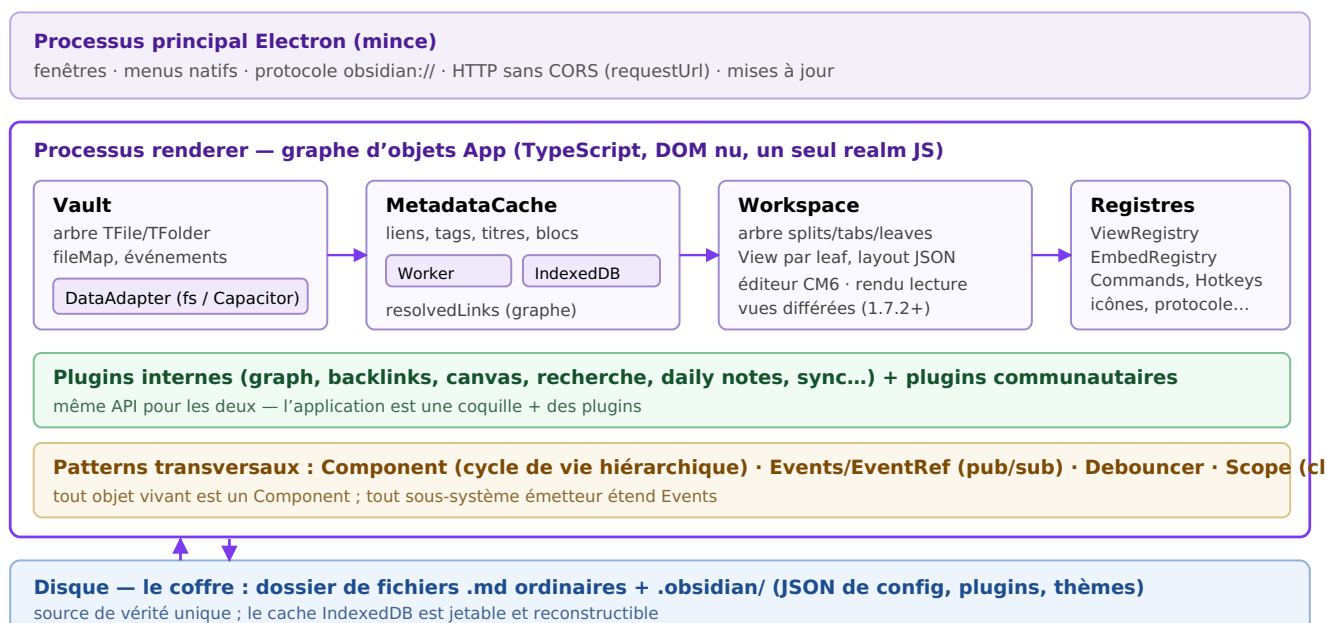


Fig. 1 — Topologie générale inférée : un processus principal minimal, un renderer qui porte tout, un disque souverain.

3. Le graphe d'objets `App` et le pattern « registres »

`App` est la racine explicite du graphe d'objets. Il n'y a ni conteneur d'injection de dépendances, ni singletons dispersés : chaque objet reçoit `app` (les plugins via leur constructeur, les vues via leur `leaf`), et navigue vers les services par propriétés. C'est un *service locator* assumé — pragmatique pour une app mono-instance, et remarquablement lisible pour l'écosystème.

3.1 Surface publique et surface réelle

L'API publique n'expose qu'une fraction d'`App` **API** : `workspace`, `vault`, `metadataCache`, `fileManager`, `keymap`, `scope`, `secretStorage`, `renderContext`, `lastEvent`, plus quelques utilitaires (`isDarkMode()`, `loadLocalStorage()` / `saveLocalStorage()` — stockage par coffre). Les typages communautaires révèlent le reste **COMM** :

Manager interne	Rôle inféré
<code>commands</code>	Registre des commandes (<code>commands</code> + <code>editorCommands</code>), exécution par id : <code>executeCommandById()</code> , <code>listCommands()</code> .
<code>hotkeyManager</code>	Raccourcis : <code>defaultKeys</code> / <code>customKeys</code> , table « compilée » <code>bakedHotkeys</code> + <code>bakedIds</code> pour un dispatch rapide (§12.5).
<code>plugins</code>	Plugins communautaires : manifestes, <code>enabledPlugins</code> : <code>Set</code> , chargement/déchargement, mises à jour, dépréciations (§11).
<code>internalPlugins</code>	Plugins « cœur » (graph, backlinks, canvas...) avec config séparée (§11.6).
<code>viewRegistry</code>	Deux tables : type de vue → fabrique, et extension de fichier → type de vue (§8.3). Émet <code>view-registered</code> , <code>extensions-updated</code> .
<code>embedRegistry</code>	Extension de fichier → fabrique de transclusion <code>![[...]]</code> (§10.5).
<code>metadataTypeManager</code>	Types des propriétés YAML (schéma de <code>types.json</code> , widgets par type) (§7.6).
<code>customCss</code>	Thèmes et snippets CSS : chargement, bascule, événement <code>css-change</code> .
<code>dragManager</code> , <code>foldManager</code> , <code>dom</code> , <code>statusBar</code> , <code>setting</code>	Glisser-déposer, mémorisation des replis, racines DOM partagées, barre d'état, modale de réglages.
<code>mobileNavbar</code> , <code>mobileToolbar</code> , <code>mobileTabSwitcher</code>	Chrome UI spécifique mobile (null sur desktop).
<code>cli</code> , <code>shareReceiver</code> , <code>appMenuBarManager</code>	Intégration CLI (API 1.12 : <code>registerCliHandler</code> API), réception de partage OS (mobile), menu natif.

3.2 Le pattern dominant : tout point d'extension est un registre

La liste ci-dessus révèle la règle de conception la plus systématique d'Obsidian : chaque famille de comportements extensibles est un **registre** — une table de correspondance mutable, interrogée au moment de l'usage, jamais figée à la compilation. Vues par type, vues par extension de fichier, embeds par extension, commandes par id, icônes par nom (`addIcon`), gestionnaires de protocole par action, post-processeurs Markdown ordonnés, extensions d'éditeur par plugin, widgets de propriétés par type, vues Bases par id. L'application « cœur » ne fait que pré-remplir ces tables ; les plugins y écrivent avec les mêmes clefs. C'est l'incarnation du principe ouvert/fermé à l'échelle d'une application entière, et la raison profonde pour laquelle un PDF, une image ou un canvas ne sont « que » des vues (§8.3).

À copier tel quel

Dans votre clone, faites des registres des objets de première classe, avec événements de modification (Obsidian : `view-registered`, `extensions-updated` **COMM**) pour que l'UI se mette à jour quand un plugin

s'enregistre après coup. Et donnez à chaque `registerX()` son symétrique automatique de désinscription — c'est l'objet de la section suivante.

4. Component : le cycle de vie hiérarchique

S'il ne fallait retenir qu'un pattern de tout ce document, ce serait celui-ci. Le danger n°1 d'une plateforme à plugins rechargeables à chaud est la fuite : listeners DOM orphelins, intervalles fantômes, abonnements jamais résiliés. Obsidian ne traite pas ce problème par la discipline, mais **par construction**, avec une classe de 13 méthodes [API](#) :

```
export class Component {
  load(): void;           // charge ce composant et ses enfants
  onload(): void;         // hook – surchargé par les sous-classes
  unload(): void;         // décharge enfants + exécute les nettoyages
  onunload(): void;       // hook symétrique
  addChild<T extends Component>(component: T): T; // lie le cycle de vie
  removeChild<T extends Component>(component: T): T;
  register(cb: () => any): void; // nettoyage arbitraire
  registerEvent(eventRef: EventRef): void; // désabonnement auto
  registerDomEvent(el, type, callback): void; // removeEventListener auto
  registerInterval(id: number): number; // clearInterval auto
}
```

Le contrat : tout ce qu'un composant acquiert pendant sa vie est déclaré via `register*` ; `unload()` rejoue la pile de nettoyages en sens inverse et se propage récursivement aux enfants (`addChild`). C'est du RAI à la sauce JavaScript. Or *tout* objet vivant d'Obsidian est un `Component` : `Plugin`, `View` (donc chaque onglet), `MarkdownRenderChild` (chaque widget rendu dans une note), `HoverPopover`, et même `Menu` [API](#). Désactiver un plugin, fermer un onglet, re-rendre une section de note : une seule mécanique de démontage, uniforme et récursive.

```
// Un plugin type : aucune ligne de nettoyage manuel, onunload() peut rester vide.
export default class MonPlugin extends Plugin {
  async onload() {
    this.registerEvent(this.app.vault.on("rename", (f, old) => this.onRename(f, old)));
    this.registerDomEvent(document, "click", (e) => this.onClick(e));
    this.registerInterval(window.setInterval(() => this.tick(), 60_000));
    this.register(() => monNettoyageArbitraire());
    this.addChild(new MonSousSysteme(this.app)); // vie et mort liées au plugin
  }
}
```

Détail d'API révélateur : les méthodes de `Plugin` comme `registerMarkdownPostProcessor` ou `registerEditorExtension` (§11.3) ne retournent pas de fonction de désinscription — elles enregistrent le nettoyage *dans* le composant. L'auteur de plugin ne peut pas oublier de nettoyer, car il n'a jamais eu la responsabilité de le faire. Les vues ajoutent deux paires de hooks au-dessus de ce socle : `onOpen()` / `onClose()` (montage DOM) et, pour les vues liées à un fichier, `onLoadFile()` / `onUnloadFile()` [API](#).

Pourquoi c'est « smart »

Le coût marginal d'un point d'extension devient quasi nul : chaque nouveau `registerX()` hérite gratuitement de la sémantique de démontage. C'est ce qui permet à Obsidian de multiplier les hooks (section 5) sans multiplier les bugs de cycle de vie. Implémentez `Component` en premier dans votre clone ; faites-en la classe mère de tout.

5. Events : le pub/sub typé et la carte complète des hooks

5.1 Le mécanisme

Le second pattern transversal est un émetteur d'événements maison, minuscule **API** :

```
export class Events {
  on(name: string, callback: (...data) => unknown, ctx?: any): EventRef;
  off(name: string, callback: (...data) => unknown): void;
  offref(ref: EventRef): void; // désinscription par référence
  trigger(name: string, ...data: unknown[]): void;
  tryTrigger(evt: EventRef, args: unknown[]): void; // dispatch défensif
}
```

Trois choix méritent l'attention. **(1)** `on()` retourne un `EventRef` opaque : la désinscription par référence (`offref`) évite le piège classique du `removeEventListener` qui exige la même identité de fonction ; et c'est précisément ce que `Component.registerEvent()` consomme — les deux patterns s'emboîtent. **(2)** Chaque émetteur concret surcharge `on()` avec des signatures typées par nom d'événement (voir tables ci-dessous) : du pub/sub stringly-typed rendu sûr à l'usage. **(3)** L'existence de `tryTrigger` indique un dispatch enveloppé de try/catch : un listener de plugin qui lève une exception ne doit pas casser la chaîne des autres listeners **INF**.

Enfin, les émetteurs sont *locaux* : `Vault`, `MetadataCache`, `Workspace`, `WorkspaceItem` étendent chacun `Events`. Il n'y a pas de bus global anonyme ; on s'abonne à l'objet dont on dépend, ce qui garde les dépendances lisibles.

5.2 La carte des hooks — ce à quoi les développeurs ont pensé

Voici l'inventaire exhaustif des événements de l'API publique **API**. Cette liste est la réponse à « quels hooks ont-ils prévus ? » : elle dessine en creux les flux internes de l'application.

Vault — la vérité du disque

Événement	Signature du callback	Émis quand
<code>create</code>	<code>(file: TAbstractFile)</code>	Fichier/dossier créé (aussi émis pour chaque fichier au chargement initial du coffre — la doc conseille de s'abonner dans <code>onLayoutReady</code> pour éviter la rafale).
<code>modify</code>	<code>(file: TAbstractFile)</code>	Contenu modifié (par l'app ou détecté sur disque).
<code>delete</code>	<code>(file: TAbstractFile)</code>	Suppression.
<code>rename</code>	<code>(file, oldPath: string)</code>	Renommage/déplacement — l'ancien chemin est fourni : indispensable pour re-clefer ses propres index.

MetadataCache — la vérité de l'index

Événement	Signature	Émis quand
<code>changed</code>	<code>(file, data: string, cache: CachedMetadata)</code>	L'index d'un fichier vient d'être (re)calculé ; livre contenu et cache — le consommateur n'a pas à relire le disque.

Événement	Signature	Émis quand
deleted	(file, prevCache: CachedMetadata null)	Fichier supprimé ; l'ancien cache est fourni pour permettre le nettoyage différentiel.
resolve	(file: TFile)	Les liens de ce fichier ont été (re)résolus vers leurs cibles.
resolved	()	Fin d'un lot de résolution : l'index global est stable (signal attendu par la vue graphe, les plugins d'analyse...).

Workspace — la vérité de l'écran

Événement	Signature	Usage
active-leaf-change	(leaf: WorkspaceLeaf null)	Focus d'onglet — le hook le plus utilisé des plugins d'UI.
file-open	(file: TFile null)	Un fichier devient le fichier actif.
layout-change	()	Split/onglet créé, déplacé, fermé.
resize	()	Géométrie des panneaux modifiée.
css-change	()	Thème/snippet rechargé — les vues qui mesurent le DOM se recalculent.
quick-preview	(file: TFile, data: string)	Contenu tapé <i>avant</i> écriture disque : permet aux vues dérivées (outline, recherche) de refléter la frappe sans attendre la sauvegarde débouncée. Hook subtil et précieux.
file-menu / files-menu	(menu: Menu, file(s), source, leaf?)	Interception des menus contextuels : le plugin <i>mute</i> l'objet Menu reçu (§5.3).
editor-menu	(menu, editor, info)	Menu contextuel dans l'éditeur.
url-menu	(menu, url: string)	Menu contextuel sur lien externe.
editor-change	(editor, info)	Frappe dans l'éditeur (relais haut niveau des transactions CM6).
editor-paste / editor-drop	(evt, editor, info)	Interception du collage/dépôt : <code>evt.preventDefault()</code> pour substituer son propre comportement (ex. : coller une URL → lien riche).
window-open / window-close	(win: WorkspaceWindow, window: Window)	Cycle de vie des popouts (§2.4).
quit	(tasks: Tasks)	Arrêt : on pousse des promesses dans <code>tasks</code> pour retarder proprement la fermeture (flush d'écritures). Pattern « graceful shutdown » explicite.

Divers

`WorkspaceLeaf` émet `pinned-change` et `group-change` ; `ViewRegistry` **COMM** émet `view-registered/view-unregistered/extensions-updated` ; `Publish` et `SecretStorage` sont aussi des `Events` **API**.

5.3 Le pattern d'interception par mutation

Les hooks `*-menu` illustrent une convention discrète : plutôt que de retourner des « contributions » déclaratives (comme les points de contribution JSON de VS Code), Obsidian passe l'objet `Menu` en construction et laisse chaque listener le muter (`menu.addItem(...)`). Même logique pour `editor-paste`

(annulable via `preventDefault`) et pour les `Tasks` de `quit`. C'est moins « pur » mais radicalement plus simple, et cohérent avec le modèle de confiance totale accordé aux plugins (§11.4).

5.4 La cascade type

Les trois tables ci-dessus s'enchaînent en une cascade que l'on retrouvera dans les flux de la section 16 : **disque** → **Vault (create/modify/delete/rename)** → **MetadataCache (changed → resolve → resolved)** → **consommateurs** (backlinks, graphe, onglets). La règle implicite : on ne s'abonne jamais « plus bas » que nécessaire. Un plugin d'affichage écoute `metadataCache.changed` (qui lui livre le cache prêt à l'empl

6. Vault & DataAdapter : la persistance

6.1 Deux étages bien séparés

La persistance est découpée en deux abstractions aux responsabilités disjointes **API**. En bas, `DataAdapter` : une interface purement « chemins et octets » (`read`, `write`, `list`, `stat`, `mkdir`, `rename`, `copy`, `trashSystem/trashLocal...`), sans aucune notion de note, de lien ou d'événement. En haut, `Vault` : un arbre *en mémoire* d'objets `TAbstractFile` (spécialisés en `TFile` — `basename`, `extension`, `stat` — et `TFolder` — `children`), doublé d'un index plat `fileMap: chemin → objet` **COMM**, qui étend `Events` et émet les quatre événements de la §5.2.

Ce découpage est un pattern *Bridge* au sens strict : la hiérarchie d'abstraction (`Vault` et tout ce qui est au-dessus) varie indépendamment de la hiérarchie d'implémentation (`FileSystemAdapter` sur Node/desktop, `CapacitorAdapter` sur mobile **API**). C'est lui qui a permis de porter Obsidian sur mobile sans toucher au cœur — et c'est lui qui permettrait un adaptateur OPFS/IndexedDB pour une version navigateur de votre clone.

6.2 Identité des objets et lectures cachées

Point de conception important : les `TFile` sont des objets à **identité stable** — le même objet est muté lors d'un renommage (l'événement `rename` fournit `oldPath` précisément parce que `file.path` a déjà changé). Les plugins peuvent donc les utiliser comme clefs. Deux lectures coexistent : `read()` (disque, pour éditer) et `cachedRead()` (copie mémoire, pour afficher/analyser) **API**, avec une limite de cache interne (`cacheLimit` **COMM**). La doc officielle recommande `cachedRead` pour tout ce qui n'écrit pas — un détail qui trahit une app conçue pour des coffres de dizaines de milliers de fichiers.

6.3 Écritures : atomicité et débouncing

Trois mécanismes se partagent la robustesse des écritures. **(1)** `Vault.process(file, fn)` **API** : lecture-transformation-écriture *atomique* vis-à-vis des autres acteurs de l'app — la réponse officielle aux plugins qui se marchaient dessus avec des `read/modify` concurrents. **(2)** `DataWriteOptions {ctime?, mtime?}` : la possibilité de forcer les horodatages, nécessaire à Sync pour rejouer des modifications sans fausser la détection de conflits. **(3)** Le débouncing généralisé : l'éditeur expose `requestSave` (sauvegarde différée ~2 s après la dernière frappe **INF**), le layout `requestSaveLayout`, chaque config `requestSaveConfig` — tous typés `Debouncer`, l'utilitaire public `debounce()` **API**. La règle : on n'écrit jamais le disque dans un chemin chaud ; on le « demande ».

6.4 Le monde extérieur écrit aussi

« Files over app » implique que Git, Dropbox ou un autre éditeur peuvent modifier le coffre à tout moment. Le `Vault` possède un handler de surveillance (`onChange: FileSystemWatchHandler` **COMM**) : les notifications du système de fichiers sont réconciliées avec `fileMap` puis réémises comme événements normaux — les couches hautes ne savent pas si une modification vient de l'app ou d'ailleurs. Même les réglages y passent : `Plugin.onExternalSettingsChange()` **API** notifie un plugin que son `data.json` a changé sur disque (typiquement via Sync). Au démarrage, la réconciliation se fait par comparaison `mtime/taille` avec l'index persisté (§7.2) **INF**.

6.5 FileManager : la couche sémantique

Au-dessus du stockage brut, FileManager API regroupe les opérations qui *comprennent* le Markdown et les liens : `renameFile()` (renomme **et met à jour tous les wikilinks entrants**, en s'appuyant sur l'index inversé du cache ; des `LinkUpdaters` spécialisés existent par format, dont `CanvasLinkUpdater` pour les fichiers `.canvas` COMM), `generateMarkdownLink()` (respecte les préférences de l'utilisateur : wikilink ou lien MD, chemin le plus court...), `processFrontMatter()` (édition atomique du YAML en place), `getAvailablePathForAttachment()`. La séparation Vault/FileManager est une leçon d'architecture : le stockage ignore la sémantique, la sémantique n'accède jamais au disque directement.

6.6 Le dossier `.obsidian/` : la config en fichiers plats

Fichier / dossier	Contenu
<code>app.json</code>	Réglages « Éditeur/Fichiers » du coffre (structure <code>AppVaultConfig</code> COMM).
<code>appearance.json</code>	Thème, police, taille, snippets actifs.
<code>core-plugins.json</code>	Activation des plugins internes.
<code>community-plugins.json</code>	Liste des ids de plugins communautaires activés (le <code>Set enabledPlugins</code> COMM).
<code>hotkeys.json</code>	Surcharges de raccourcis (<code>customKeys</code> du <code>HotkeyManager</code>).
<code>workspace.json</code>	Sérialisation de l'arbre Workspace (§8.2).
<code>graph.json</code> , <code>types.json</code> , <code>bookmarks.json</code> ...	Chaque plugin interne persiste son propre JSON.
<code>plugins/<id>/</code>	<code>manifest.json</code> + <code>main.js</code> + <code>styles.css</code> + <code>data.json</code> (réglages du plugin, via <code>loadData()</code> / <code>saveData()</code>).
<code>themes/<nom>/</code> , <code>snippets/</code>	<code>manifest.json</code> + <code>theme.css</code> ; snippets CSS individuels.

Aucune base de registre, aucun format binaire : des JSON indépendants, sauvegardés avec *debounce*, surveillés pour les changements externes, et donc synchronisables/versionnables fichier par fichier. Le `configDir` est d'ailleurs relocalisable (`Vault.configDir` API).

7. MetadataCache : l'index incrémental

C'est le sous-système le plus « intelligent » de l'application : tout ce qui fait la valeur d'Obsidian (backlinks, graphe, recherche structurée, transclusions, propriétés) est une lecture de cet index. Sa structure interne est presque entièrement visible dans les typages communautaires [COMM](#).

7.1 Ce qui est indexé : CachedMetadata

Pour chaque fichier Markdown, l'index conserve [API](#) : `links` (wikilinks sortants), `embeds` (transclusions), `tags`, `headings`, `sections` (le découpage en blocs de premier niveau : paragraphes, listes, code...), `listItems` (avec hiérarchie parent et état de tâche), `frontmatter` (+ `frontmatterPosition`, `frontmatterLinks`), `blocks` (ancres `^block-id`), `footnotes` et `referenceLinks`. Chaque entrée est un `CacheItem` portant une position précise `Pos {start, end}` où chaque `Loc` donne `{line, col, offset}`. Ces offsets absolus sont la clef des éditions chirurgicales sans re-parsing : renommer un titre, cocher une tâche, réécrire un lien = un `replaceRange` aux offsets du cache.

7.2 Le pipeline : worker, adressage par contenu, IndexedDB

Les champs internes racontent le pipeline complet [COMM](#) :

```
interface MetadataCache extends Events {
  worker: Worker; // parsing Markdown hors thread principal
  workQueue: PromisedQueue; // sérialisation des tâches d'indexation
  fileCache: { [path: string]: { hash, mtime, size } }; // chemin → empreinte
  metadataCache: { [hash: string]: CachedMetadata }; // hash de contenu → index
  db: IDBDatabase; // persistance IndexedDB (démarrage à chaud)
  linkResolverQueue: ItemQueue<TFile>; // résolution différée des liens
  resolvedLinks: { [src: string]: { [cible: string]: nombre } };
  unresolvedLinks: { [src: string]: { [cible: string]: nombre } };
  uniqueFileLookup: CustomArrayDict<TFile>; // nom de fichier → candidats
  userIgnoreFilters: RegExp[] | null; // motifs d'exclusion
  onCleanCacheCallbacks: (() => void)[]; // « préviens-moi quand l'index est stable »
}
```

Lecture de ce design, étage par étage :

- **Parsing en Web Worker.** Le Markdown est analysé hors du thread UI (message de retour `{data: CachedMetadata}` [COMM](#)) ; une `PromisedQueue` sérialise les demandes. L'indexation initiale d'un gros coffre ne gèle donc jamais l'interface — condition de survie du « premier lancement » sur un coffre de 20 000 notes.
- **Cache adressé par contenu.** La double table est le choix le plus fin : `fileCache` associe chaque *chemin* à une empreinte `{hash, mtime, size}`, et `metadataCache` associe le *hash* au résultat de parsing. Conséquences : un renommage ne coûte rien (le hash ne change pas, l'index est retrouvé) ; des fichiers au contenu identique partagent une entrée ; l'invalidation se décide sur `mtime/size` sans lire le fichier, avec le hash comme arbitre en cas de doute.
- **Persistance IndexedDB.** L'aide officielle le confirme : « le cache est stocké dans IndexedDB et reconstruit automatiquement à partir de vos fichiers » [API](#). Au boot, `_preload()` recharge l'index et seuls les fichiers dont l'empreinte a changé repassent par le worker. Le cache reste un *cache* : sa corruption n'est jamais une perte de données.
- **Résolution des liens en seconde passe.** Parser un fichier dit « ce fichier référence `[[Idée]]` » ; savoir *quel fichier est* « Idée » dépend de tout le coffre. D'où la `linkResolverQueue` : après le parsing, une passe de résolution transforme les textes de liens en chemins cibles, alimente `resolvedLinks/unresolvedLinks`, émet `resolve` par fichier puis `resolved` quand le lot est fini. `uniqueFileLookup`

(multi-dictionnaire nom → fichiers) sert l'algorithme du « chemin le plus court » des wikilinks et le quick switcher.

7.3 Le graphe est déjà là

`resolvedLinks` est une liste d'adjacence pondérée couvrant tout le coffre : `source → {cible → nombre d'occurrences}`. La vue graphe (§13) et le panneau de backlinks ne construisent **aucun** index propre : le graphe de connaissances est un sous-produit permanent de l'indexation. `unresolvedLinks` — les liens vers des notes qui n'existent pas encore — donne les « nœuds fantômes » du graphe et le cœur du workflow Zettelkasten (créer la note en cliquant le lien mort).

7.4 L'API de consommation

Côté public **API** : `getFileCache(file)/getCache(path)` (l'index d'un fichier), `getFirstLinkpathDest(linkpath, sourcePath)` (résolution d'un wikilink depuis un contexte), `fileToLinktext()` (l'inverse), et les helpers `resolveSubpath()` (cible `#titre` ou `#^bloc` dans un fichier), `getAllTags()`, `getFrontmatterInfo()`. Le pattern d'usage canonique d'un plugin : écouter `changed`, lire le cache, ne jamais re-parser.

7.5 Files d'attente et signaux de stabilité

Trois dispositifs anti-tempête **COMM** : `didFinish: Debouncer` (l'événement `resolved` n'est émis qu'après accalmie), `transactionSave: Debouncer` (les écritures IndexedDB sont groupées), `onCleanCache(cb)` (rappel unique « quand l'index est propre » — le signal qu'attendent les plugins de statistiques avant de balayer le coffre). À retenir pour un clone : chaque frontière de sous-système a sa politique de coalescence explicite.

7.6 MetadataTypeManager : le schéma des propriétés

Depuis les « Properties » (v1.4), les clefs de frontmatter ont un type (texte, nombre, date, liste, case à cocher...) géré par un manager dédié **COMM**, persisté dans `types.json`, avec un registre de widgets d'édition par type — encore un registre (§3.2). Le sous-système « Bases » (v1.9+ : vues tabulaires requêtables sur les propriétés, fichiers `.base`) s'appuie dessus et expose dans l'API publique tout un système de valeurs typées (`Value → StringValue/NumberValue/DateValue/ListValue...`) et de formules (`FormulaContext`, `BasesEntry`, `QueryController`) **API** — signe d'une évolution vers une couche « base de données » au-dessus du même cache.

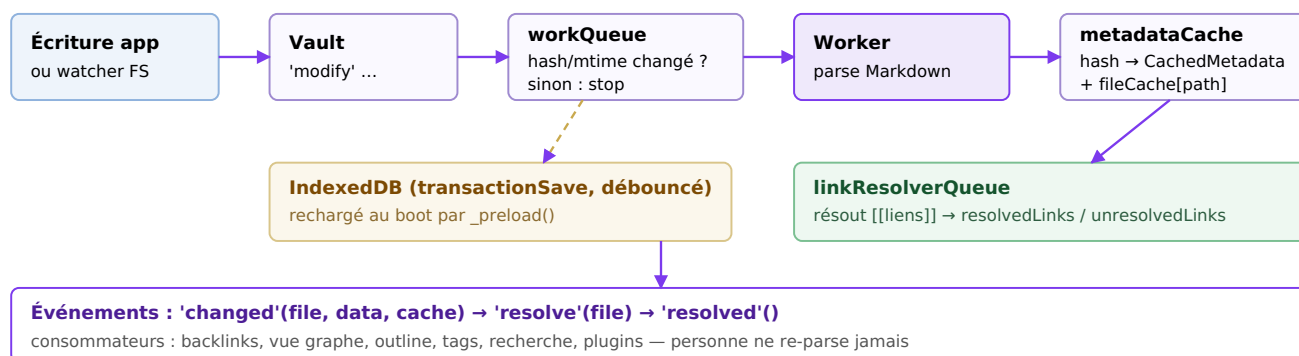


Fig. 2 — Pipeline d'indexation inféré : coalescence, worker, cache adressé par contenu, persistance déboucée, résolution en seconde passe.

8. Workspace : fenêtrage composite et vues différées

8.1 Un arbre composite dont chaque nœud est un émetteur

Le fenêtrage est un pattern *Composite* canonique **API** : `WorkspaceItem` (qui étend `Events`) est la base ; `WorkspaceParent` porte des enfants ; `WorkspaceSplit` divise horizontalement/verticalement ; `WorkspaceTabs` empile des onglets ; `WorkspaceLeaf` est la feuille — et une feuille héberge exactement une `View`. Les conteneurs racine spécialisent l'arbre par contexte : `WorkspaceRoot` (fenêtre principale), `WorkspaceWindow` (popout), `WorkspaceSidedock` (panneaux latéraux desktop), `WorkspaceMobileDrawer` (tiroirs mobile), `WorkspaceFloating`. Le `Workspace` lui-même expose `rootSplit`, `leftSplit/rightSplit`, `leftRibbon`, et toute une algèbre de navigation : `getLeaf('tab'|'split'|'window')`, `iterateAllLeaves`, `getLeavesOfType`, `getMostRecentLeaf` (MRU via `activeTime` **COMM**), `revealLeaf`, `ensureSideLeaf`.

8.2 L'état en deux couches : persistant et éphémère

Chaque vue sait se sérialiser : `getState()/setState()` produit un objet JSON (`ViewState {type, state}`) — pour un onglet Markdown : fichier et mode. À côté, `getEphemeralState()/setEphemeralState()` porte ce qui doit survivre à la navigation mais pas forcément au redémarrage : défilement, sélection **API**. Le `Workspace` agrège le tout (`getLayout()/changeLayout()`) dans `workspace.json`, sauvegardé par `requestSaveLayout` débouncé. Cette dualité état/état-éphémère alimente trois fonctions d'un coup : restauration après crash, historique par onglet (chaque `WorkspaceLeaf` a un `history` avec pile avant/arrière d'états **COMM**), et onglets liés (`setGroup` : les feuilles d'un groupe synchronisent leur navigation **API**).

8.3 ViewRegistry : le type de fichier est une vue

Le registre des vues tient en deux tables **COMM** : `viewByType` (`type → fabrique (leaf) => View`) et `typeByExtension` (`extension → type`). Ouvrir `photo.png`, c'est : `extension → type image → fabrique → vue` dans la feuille. Un plugin fait exactement pareil avec `registerView()` + `registerExtensions()` **API**. PDF (via `pdf.js`), audio, images, `.canvas`, `.base` : aucune n'est câblée en dur — toutes passent par le registre. La hiérarchie fournit des bases toutes prêtes selon le degré de liaison au fichier :

```
View // n'importe quel panneau (recherche, outline...)
├─ ItemView // + conteneur standard, icône, actions d'en-tête
│   └─ FileView // + liaison à un TFile (onLoadFile/onUnloadFile)
│       └─ EditableFileView
│           └─ TextFileView // + data:string, requestSave débouncé, save()
│               └─ MarkdownView // compose éditeur CM6 + previewMode
```

8.4 DeferredView : la paresse comme optimisation majeure (v1.7.2)

Depuis 1.7.2, **toutes** les vues sont créées au démarrage comme `DeferredView` : des coquilles vides qui ne deviennent la « vraie » vue qu'au moment où l'onglet devient visible **API** (doc officielle « Deferred views »). Le layout complet — 40 onglets, 3 fenêtres — est restauré instantanément, mais seul ce qui est à l'écran paie son coût d'instanciation. L'API d'accompagnement : `leaf.isDeferred`, `await leaf.loadIfDeferred()`, et une discipline nouvelle pour les plugins : tester `leaf.view instanceof MaVue` et non `getViewType()`, car `leaf.view` peut être le placeholder. C'est le compromis classique proxy/lazy-loading, appliqué tardivement mais radicalement — un clone peut l'intégrer dès le premier jour à moindre coût.

Chronologie utile

Le CHANGELOG de l'API montre la même préoccupation de démarrage partout : vues différées (1.7.2), `onUserEnable()` pour éviter du travail au chargement, lazy-loading de Prism/MathJax/Mermaid/PDF.js (§10.3), cache IndexedDB (§7.2). La performance de démarrage est traitée comme une *feature* de premier rang.

8.5 Navigation et ouverture de liens

Le chemin « clic sur un wikilink » est entièrement public : `workspace.openLinkText(linktext, sourcePath, newLeaf?)` résout via le cache (§7.4), choisit la feuille selon `PaneType` (`'tab'|'split'|"window"`), et `Keymap.isModEvent(evt)` convertit uniformément l'état des modificateurs (Ctrl/Cmd-clic → onglet, etc.) en `PaneType` **API** — un seul helper pour une convention d'UX appliquée partout. `OpenViewState` permet de cibler un état (ligne, mode) à l'ouverture.

9. L'éditeur : CodeMirror 6 sous une façade

9.1 La façade Editor : une couche anti-corruption

L'éditeur est CodeMirror 6 (imports `@codemirror/state` et `@codemirror/view` en tête de `obsidian.d.ts` **API**, grammaire Lezer **CRED**, mode Vim **CRED**). Mais l'API plugin ne vous donne pas CM6 : elle donne Editor, une classe abstraite au vocabulaire de CodeMirror 5 (`getCursor`, `getLine`, `replaceRange`, `posToOffset/offsetToPos`, `transaction()`... jusqu'au vestige `getDoc(): this`) **API**. L'histoire s'y lit : Obsidian a migré CM5 → CM6 (v0.13, « Live Preview ») en préservant l'écosystème — les plugins écrits contre la façade ont survécu au remplacement complet du moteur. C'est un cas d'école de couche anti-corruption : la dépendance la plus volatile du projet (le moteur d'édition) est la mieux encapsulée.

9.2 L'injection d'extensions et les ponts d'état

Pour qui veut la puissance de CM6, l'accès existe, mais canalisé : `plugin.registerEditorExtension(extension)` injecte des extensions CM6 dans **tous** les éditeurs du coffre, dynamiquement (reconfiguration à chaud via `workspace.updateOptions()` **API** ; mécanique de Compartiment CM6 **INF**). Dans l'autre sens, Obsidian publie des `StateField` qui exposent le contexte hôte à l'intérieur de l'état CM6 **API** :

- `editorEditorField: StateField<EditorView>` — la vue CM courante ;
- `editorInfoField: StateField<MarkdownFileInfo>` — le fichier/la vue Obsidian propriétaire ;
- `editorLivePreviewField: StateField<boolean>` — mode Live Preview ou source ;
- `livePreviewState: ViewPlugin` — micro-états d'interaction (souris enfoncée...).

Une extension CM6 reste ainsi « pure CodeMirror » (testable hors Obsidian) tout en lisant son contexte via des facilités standard. C'est le bon sens du couplage : l'hôte se projette dans le composant, jamais l'inverse.

9.3 Live Preview : du WYSIWYG sans quitter le mode source

Le Live Preview n'est *pas* un rendu HTML séparé : c'est le document source dans CM6, habillé de *décorations* calculées depuis l'arbre syntaxique Lezer — les marqueurs (`**`, `[]...`) sont masqués sauf lorsque la sélection les touche (« reveal on cursor »), et les objets non textuels (images, transclusions, blocs de code rendus, LaTeX) sont des widgets de remplacement insérés dans le flux de l'éditeur **INF** (corroboré par l'analyse communautaire : DOM du Live Preview ≈ DOM du mode source, jetons inactifs remplacés). L'avantage décisif sur un WYSIWYG classique : une seule source de vérité (le texte), pas de conversion aller-retour, et le mode source reste disponible gratuitement. `MarkdownView` compose ce sous-éditeur (`MarkdownEditView` **API**) avec le mode lecture (`MarkdownPreviewView`) derrière une interface commune `MarkdownSubView`, et bascule `currentMode` entre les deux.

9.4 Le framework de suggestions

Toutes les complétions passent par une même famille **API** : `PopoverSuggest` (popup positionné, clavier, `ISuggestOwner`) se spécialise en `EditorSuggest<T>` — le contrat en quatre temps : `onTrigger(cursor, editor, file) → {start, end, query} | null`, puis `getSuggestions(ctx)`, `renderSuggestion(item, el)`, `selectSuggestion(item)` — et en `AbstractInputSuggest` pour les champs de formulaire. Les palettes modales (`SuggestModal`, `FuzzySuggestModal`) partagent le même vocabulaire. Un plugin qui ajoute une complétion `[]` personnalisée déclare *où* il se déclenche et *quoi* proposer ; le positionnement, le clavier

et le cycle de vie sont fournis. C'est l'exemple type du « framework inversé » : l'API impose le squelette, le plugin remplit les trous.

10. Le rendu Markdown en mode lecture

10.1 Deux parseurs, deux usages

Fait architectural notable : Obsidian maintient **deux** pipelines Markdown. L'éditeur parse avec Lezer (incrémental, tolérant, orienté surlignage — jamais d'AST complet). Le mode lecture, lui, produit du HTML via un parseur de la lignée remark **CRED** (les crédits mentionnent aussi le copyright de marked, suggérant un fork hybride historique **INF**), largement personnalisé (wikilinks, transclusions, callouts, tâches...). Les deux doivent rester d'accord sur la grammaire — c'est un coût réel et assumé, car les besoins sont irréconciliables : l'un doit re-parser à chaque frappe une région locale, l'autre doit produire un DOM riche et transformable.

10.2 La chaîne de post-processeurs

Le HTML rendu passe ensuite dans une chaîne ordonnée de `MarkdownPostProcessor` (tri par `sortOrder`) **API** : chacun reçoit l'élément d'une *section* rendue et un contexte `MarkdownPostProcessorContext` qui fournit `sourcePath`, le `frontmatter`, `getSectionInfo(el)` (→ texte source et lignes de la section) et surtout `addChild(MarkdownRenderChild)` : tout enrichissement DOM devient un `Component` attaché au cycle de vie du rendu — détruit et reconstruit proprement quand la section se re-rend. Le raccourci le plus fécond de l'écosystème est `registerMarkdownCodeBlockProcessor(lang, handler)` : un bloc ```kanban` ou ```dataview` devient une zone d'UI arbitraire, tout en restant du texte brut dans le fichier — la compatibilité « files over app » est préservée par construction.

10.3 Rendu par sections et paresse systématique

Le renderer travaille par *sections* (les blocs du cache, §7.1) : c'est l'unité de re-rendu (`rerender(full?)`) **API** et de virtualisation — les longues notes ne matérialisent que le voisinage visible **INF** (fortement corroboré par l'API : `getSectionInfo` n'aurait pas de sens sinon, et le comportement est observable). Les moteurs lourds sont chargés à la demande **API** : `loadPrism()` (coloration), `loadMathJax()`, `loadMermaid()`, `loadPdfJs()` — le démarrage ne paie jamais pour ce que la note n'utilise pas. La sécurité est assurée par `DOMPurify` **CRED** : le HTML embarqué dans le Markdown est assaini avant insertion **INF**.

10.4 MarkdownRenderer : le rendu comme service

Le rendu est réutilisable partout : `MarkdownRenderer.render(app, markdown, el, sourcePath, component)` **API** — la popup de survol (page preview), les tooltips, l'export PDF, les vues de plugins rendent du Markdown avec exactement le même pipeline. Le paramètre `component` obligatoire rattache les enfants du rendu à un cycle de vie (§4) : même en usage « service », pas de fuite possible.

10.5 EmbedRegistry : les transclusions par extension

Symétrique du `ViewRegistry` pour l'intérieur des notes **COMM** : `!![[fichier.ext]]` consulte `embedByExtension` pour fabriquer le widget d'embed (image, audio, PDF, note transcluse, canvas...). Un plugin qui enregistre un nouveau type de fichier fournit donc potentiellement trois briques : une `View` (onglet), un `embed` (transclusion), et un post-processeur (rendu en lecture) — trois registres, une cohérence.

11. Le système de plugins

11.1 Anatomie et distribution

Un plugin est un dossier `.obsidian/plugins/<id>/` de trois fichiers **API** : `manifest.json` (`id`, `name`, `version`, `minAppVersion`, `description`, `author`, `isDesktopOnly`...), `main.js` (bundle CommonJS unique), `styles.css` optionnel ; plus `data.json` écrit par `saveData()`. La distribution est d'une frugalité remarquable : pas de store serveur — un dépôt Git communautaire liste les plugins approuvés, et l'app télécharge les fichiers depuis les *releases GitHub* du développeur (`installPlugin(repo, version, manifest)` **COMM**) ; `versions.json` mappe `version` → `minAppVersion` pour servir les vieux clients.

11.2 Le chargement, reconstitué

Le manager Plugins **COMM** lit `community-plugins.json` (→ `enabledPlugins: Set<string>`), charge tous les manifestes (`loadManifests`), vérifie `minAppVersion` et les dépréciations (`checkForDeprecations`, listes de plugins malveillants/cassés poussées par l'éditeur), puis pour chaque plugin activé : `loadPlugin(id)` lit `main.js` *via le Vault adapter*, l'évalue comme module CommonJS avec un `require shim` qui fournit les modules hôtes — `"obsidian"`, `"electron"`, `"@codemirror/*"`, `"moment"`... (c'est pourquoi la config esbuild officielle des plugins marque ces imports en `external` **API**) — instancie la classe exportée par défaut avec (`app`, `manifest`), injecte `styles.css`, et appelle `plugin.load()` → votre `onload()` **INF** (mécanique confirmée par l'observation ; seuls les détails d'évaluation restent opaques). La désactivation est l'exact miroir : `unload()` rejoue tous les `register*` (§4).

11.3 La surface d'enregistrement — inventaire complet

Méthode de Plugin API	Point d'extension
<code>addCommand</code> / <code>removeCommand</code>	Palette de commandes, raccourcis ; variantes <code>checkCallback</code> (§11.5).
<code>addRibbonIcon</code>	Barre d'icônes latérale.
<code>addStatusBarItem</code>	Barre d'état (desktop).
<code>addSettingTab</code>	Onglet de réglages du plugin (§11.7).
<code>registerView</code> + <code>registerExtensions</code>	Nouveau type d'onglet ; association d'extensions de fichiers (§8.3).
<code>registerMarkdownPostProcessor</code>	Transformation du HTML rendu (§10.2).
<code>registerMarkdownCodeBlockProcessor</code>	Blocs <code>```lang</code> → UI riche (§10.2).
<code>registerEditorExtension</code>	Extensions CodeMirror 6 natives (§9.2).
<code>registerEditorSuggest</code>	Complétion dans l'éditeur (§9.4).
<code>registerHoverLinkSource</code>	Participation au « page preview » au survol (§12.4).
<code>registerObsidianProtocolHandler</code>	Actions <code>obsidian://<action>?params</code> — automatisation inter-applications.
<code>registerCliHandler</code>	Sous-commandes de la CLI Obsidian (nouveau 1.12).
<code>registerBasesView</code>	Nouveaux types de vues Bases (tableaux, kanban...) (§7.6).
<code>loadData</code> / <code>saveData</code>	Réglages persistants (<code>data.json</code>).
hérités de <code>Component</code> : <code>registerEvent</code> , <code>registerDomEvent</code> , <code>registerInterval</code> , <code>register</code> , <code>addChild</code>	Toute ressource arbitraire, avec nettoyage automatique (§4).

11.4 Le modèle de confiance : pas de sandbox, et c'est un choix

Les plugins s'exécutent avec les pleins pouvoirs du renderer — DOM, réseau, et Node/Electron sur desktop. Aucune isolation type iframe/worker/permissions. Les contreparties institutionnelles : « mode restreint » par défaut (plugins communautaires désactivés tant que l'utilisateur ne les autorise pas), revue humaine à l'entrée du catalogue, mécanisme de dépréciation d'urgence **COMM**, et transparence (code source public exigé). Ce pari — puissance maximale contre confiance outillée — explique directement la richesse de l'écosystème (Dataview, Excalidraw, Templater n'existeraient pas dans une sandbox à permissions) et la culture du monkey-patching : faute de hook officiel, les plugins patchent couramment les méthodes des objets vivants (bibliothèque communautaire `monkey-around`), ce que l'absence de sandbox rend possible. Pour un clone, c'est LA décision de gouvernance à prendre tôt : elle conditionne l'API, la sécurité et l'écosystème.

11.5 Les commandes : le micro-pattern `checkCallback`

```
this.addCommand({
  id: "exporter-note",           // préfixé par l'id du plugin
  name: "Exporter la note active",
  editorCheckCallback: (checking, editor, ctx) => {
    if (!peutExporter(ctx)) return false; // commande cachée/grisée
    if (!checking) exporter(editor, ctx); // exécution réelle
    return true;
  },
});
```

Le paramètre `checking` fusionne « la commande est-elle disponible ? » et « exécute-la » en une seule fonction sans effets de bord au premier appel **API** — la palette interroge la disponibilité en masse, sans coût. Les variantes `editorCallback`/`editorCheckCallback` injectent le contexte d'édition. Côté raccourcis, le `HotkeyManager` « compile » (*bake*) défauts + surcharges en une table plate `bakedHotkeys/bakedIds` **COMM** pour un dispatch clavier $O(1)$, recompilée à chaque changement — un pattern de dénormalisation volontaire.

11.6 Les plugins internes : l'app se mange elle-même

Les typages communautaires listent ~30 `InternalPlugin` **COMM** : explorateur de fichiers, recherche, quick switcher, graphe, backlinks, outline, tags, canvas, bookmarks, daily notes, templates, propriétés, page preview, slides (via `reveal.js` **CRED**), enregistreur audio, récupération de fichiers, importateur, Sync, Publish, Bases... Chacun suit le même cycle de vie que les plugins communautaires (avec une nuance : une couche `InternalPluginInstance` et une config `core-plugins.json` séparée). Ce *dogfooding* est le meilleur garant de la qualité de l'API : si le graphe et la recherche se construisent avec les hooks publics, les hooks publics suffisent pour presque tout. Un clone devrait s'imposer la même règle dès le premier sprint : **aucune feature cœur câblée hors du système de plugins**.

11.7 Réglages : du builder fluide au déclaratif

L'UI de réglages s'écrit avec un builder impératif **API** : `new Setting(containerEl).setName(...).setDesc(...).addToggle(t => t.setValue(v).onChange(...))` — une vingtaine de contrôles chaînables (§12.2). L'API récente ajoute un système *déclaratif* complet (`SettingDefinition`, groupes, pages, listes, contrôles typés par schéma **API**) : la définition JSON devient interrogeable — ce qui a permis la recherche dans les réglages. Trajectoire classique et instructive : l'impératif d'abord (rapide à livrer), le déclaratif ensuite (quand l'outillage — recherche, sync, UI générée — le justifie).

12. Le toolkit UI maison

12.1 Les prototypes DOM étendus : l'anti-framework

Au lieu d'un framework, Obsidian augmente les prototypes natifs **API** : `Node.createEl(tag, {cls, text, attr, href...}, cb)`, `createDiv`, `createSpan`, `createSvg`, `el.empty()`, `el.detach()`, `el.setText()`, `addClass/removeClass/toggleClass`, `onClickListener` — et même `Array.prototype.first/last/contains/remove/unique`, `Object.isEmpty/each`, `Math.clamp`, `String.format`. C'est un choix radical (la pollution de prototypes est un anti-pattern classique : collisions potentielles avec l'évolution du standard), mais rationnel ici : l'app est la plateforme — elle contrôle son unique realm, ne cohabite avec aucune autre bibliothèque imposée, et offre en échange une ergonomie sans imports que tous les plugins adoptent. Notez les touches multi-fenêtres : `el.doc`, `el.win`, `el.constructorWin`, `el.onWindowMigrated(cb)` et `instanceOf()` insensible aux realms (§2.4).

12.2 Les contrôles : une hiérarchie de 20 classes

Le « framework » de formulaires tient en une hiérarchie courte **API** : `BaseComponent` (activation, chaînage `then()`) → `ValueComponent<T>` (`getValue/setValue`) → `TextComponent`, `TextAreaComponent`, `ToggleComponent`, `SliderComponent`, `DropdownComponent`, `ColorComponent`, `SearchComponent`, `ProgressBarComponent`, `MomentFormatComponent` (aperçu de format de date en direct — détail révélateur du soin UX), `ButtonComponent`, `ExtraButtonComponent`, `SecretComponent`... Des objets impératifs minuscules, composés par le builder `Setting` (§11.7). Aucune réactivité magique : `onChange(cb)` et c'est tout.

12.3 Modales, menus, notifications

`Modal` (avec `titleEl/contentEl`, hooks `onOpen/onClose`), `SuggestModal<T>` et `FuzzySuggestModal<T>` (les palettes : quick switcher et palette de commandes en sont **INF**), `Menu/MenuItem` (builder, sections, sous-menus), `Notice` (toasts empilés, durée, mutable après coup) **API**. Détail mobile discret : `Modal`, `Menu` et `PopoverSuggest` implémentent `HistoryHandler` — le bouton « retour » d'Android ferme le bon élément superposé. La navigation par pile est pensée dans le socle, pas bricolée après.

12.4 Le survol universel : `HoverPopover`

Le « page preview » (aperçu d'une note au survol d'un lien) est généralisé par un contrat à trois pièces **API** : l'interface `HoverParent` (tout composant capable d'accueillir un popover — les vues, l'éditeur, le graphe l'implémentent), la classe `HoverPopover` extends `Component`, et le registre `registerHoverLinkSource(id, info)` par lequel un plugin déclare que ses liens participent au survol. L'événement interne `hover-link` fait le pont. Résultat : une seule implémentation d'aperçu pour toute l'application, extensible par déclaration.

12.5 Clavier : la pile de `Scope`

La gestion clavier est une pile de portées **API** : `Keymap.pushScope/popScope` ; chaque `Scope` enregistre des combinaisons (`scope.register(['Mod'], 'K', handler)` — 'Mod' abstrait `Cmd/Ctrl` selon l'OS) et peut chaîner un parent (fallback). Une modale qui s'ouvre pousse son scope — capture Échap, flèches, Entrée — puis le retire à la fermeture ; chaque `View` possède un `scope` optionnel activé au focus. Retourner `false` d'un handler stoppe la propagation. Combiné au « baking » du `HotkeyManager` (§11.5), le routage clavier est à la fois contextuel et rapide.

12.6 Icônes et thèmes

Les icônes sont Lucide, embarquées **CRED**, derrière un registre chaîne → SVG (`addIcon`, `setIcon`, `getIconIds` **API**). Le theming est du CSS pur : des centaines de variables CSS documentées, classes `theme-dark/theme-light` sur `body`, thèmes = un `theme.css`, snippets à chaud, événement `css-change` pour les vues qui mesurent le DOM, manager `customCss` **COMM**. L'absence de framework rend ce contrat trivial : le DOM est stable, les thèmes stylent des classes sémantiques.

13. La vue graphe : PixiJS + simulation en worker

La vue graphe est le composant le plus « moteur de jeu » de l'application. Les typages communautaires en donnent une coupe nette **COMM**, séparée en deux objets aux responsabilités disjointes :

Objet	Responsabilité
GraphEngine	L'état et la logique : options d'affichage, filtres (requête de recherche <code>fileFilter</code>), groupes de couleurs par requête, paramètres de forces (<code>centerStrength</code> , <code>repelStrength</code> , <code>linkDistance</code> ...), progression temporelle (« <code>timelapse</code> »). Construit les données depuis <code>resolvedLinks</code> / <code>unresolvedLinks</code> (§7.3), tags et pièces jointes selon les filtres.
GraphRenderer	Le rendu : <code>px: PIXI.Application</code> (WebGL CRED), un conteneur racine <code>hanger</code> (zoom/pan par transformation de scène), tableaux <code>nodes/links</code> + <code>nodeLookup</code> , canvas d'interaction séparé (<code>interactiveEl</code>), et — décisif — <code>worker: Worker</code> , documenté « thread exécutant la simulation du graphe ».

L'architecture de performance se lit dans trois champs. **La simulation de forces tourne dans un Web Worker** : le thread UI ne calcule jamais la physique, il reçoit des positions. **Les positions sont interpolées** (le zoom est décrit comme « interpolé entre la valeur précédente et `targetScale` ») : le rendu reste fluide même si la simulation cadence moins vite — découplage classique simulation/rendu des moteurs de jeu. **idleFrames compte les trames stables** : quand plus rien ne bouge, la boucle de rendu se met en veille — un graphe ouvert ne chauffe pas la machine. La bibliothèque `rbush` (R-tree spatial **CRED**) sert vraisemblablement le hit-testing et le culling des étiquettes **INF**. PixiJS est d'ailleurs exposé en global (`var PIXI` **COMM**), réutilisé par des plugins (graphes 3D, jeux...).

Le point essentiel

Le graphe ne possède aucune donnée : c'est une *projection réactive* de l'index (§7.3) filtrée par le moteur de recherche. Dans votre clone, résistez à la tentation d'un « graph store » dédié : dérivez tout de l'index de liens, et le graphe restera juste par construction.

14. Recherche et correspondance floue

L'API publique expose le noyau de correspondance **API** : `prepareFuzzySearch(query)` et `prepareSimpleSearch(query)` retournent une *fermeture compilée* — la requête est analysée une fois, puis appliquée à des milliers de candidats (pattern « prepared query », analogue aux requêtes préparées SQL). Le résultat, `SearchResult {score, matches: [début, fin][]}`, porte les offsets exacts des fragments appariés : le surlignage est exact et gratuit (`renderMatches()`/`renderResults()` les consomment), et `sortSearchResults()` trie par pertinence. `FuzzySuggestModal` branche ce noyau sur la modale de suggestion — quick switcher et palette de commandes ne sont que des instances de ce couple.

La recherche globale (opérateurs `path:`, `tag:`, `file:`, `line:`, `section:`, `task:`, `regex...`) est un plugin interne : les opérateurs structurels s'évaluent contre le `MetadataCache`, le texte intégral se balaie via `cachedRead` **INF** — il n'y a pas d'index inversé plein-texte dans le cœur (les plugins comme `Omnisearch` en ajoutent un). Compromis cohérent : l'index structurel est toujours frais et suffit aux usages majoritaires ; le plein texte paie un scan, borné par le cache mémoire.

15. Réseau, secrets, sécurité

Réseau. `requestUrl()` API est documenté « comme `fetch()`, sans aucune restriction CORS » : la requête part du côté privilégié (processus principal/couche native), pas du contexte web — indispensable aux plugins qui parlent à des API tierces sans proxy. La réponse est paresseuse (`.json`, `.text`, `.arrayBuffer`).

Secrets. `SecretStorage` extends `Events` API (récent, 1.12) centralise les jetons des plugins hors des `data.json` en clair — adossé au trousseau OS via Electron `safeStorage` INF. Le contrôle `SecretComponent` l'accompagne côté réglages.

Assainissement. `DOMPurify` CRED filtre le HTML rendu (§10.3) ; `htmlToMarkdown()` API (`Turndown` CRED) convertit le HTML collé en Markdown — le presse-papiers est traité comme une entrée non fiable à normaliser.

Chiffrement. La présence de `scrypt` dans les crédits CRED corrobore l'architecture publiée d'Obsidian Sync : dérivation de clef côté client, chiffrement de bout en bout (AES) — le serveur ne voit que des blobs. Le service est lui-même un plugin interne (§11.6) : la sync n'a aucun privilège architectural, elle écrit dans le Vault comme tout le monde (d'où `DataWriteOptions.mtime`, §6.3, et `onExternalSettingsChange`, §6.4).

Protocole. `obsidian://` route vers les `ObsidianProtocolHandler` enregistrés (`open`, `new`, `search...` + actions de plugins) API : l'automatisation externe (Raycast, scripts, liens profonds) sans serveur local.

16. Les flux critiques, pas à pas

16.1 Démarrage (reconstitution INF appuyée sur §7-§11)

1. Electron crée la fenêtre → charge le bundle app.js → new App(adapter, appId)
2. Vault : énumération du coffre via l'adapter → construction de l'arbre TFile/TFolder + fileMap ; démarrage du watcher FS
3. MetadataCache._preload() : chargement de l'index depuis IndexedDB ; diff mtime/size → seuls les fichiers modifiés repartent au Worker
4. Workspace.changeLayout(workspace.json) : reconstruction de l'arbre de fenêtrage, chaque onglet = DeferredView (coquille) → layoutReady = true → onLayoutReady()
5. internalPlugins.enable() puis plugins.initialize() : manifestes → enabledPlugins → loadPlugin(id) → onload() de chaque plugin
6. La vue active (seule) est matérialisée ; linkResolverQueue draine → 'resolve' x n → 'resolved' : l'app est « chaude »

Les invariants offerts aux plugins : ne pas toucher au layout avant `onLayoutReady` ; l'index n'est complet qu'à `resolved` ; les rafales de `create` initiales s'évitent en s'abonnant après `layoutReady`.

16.2 Une frappe clavier, du DOM au graphe

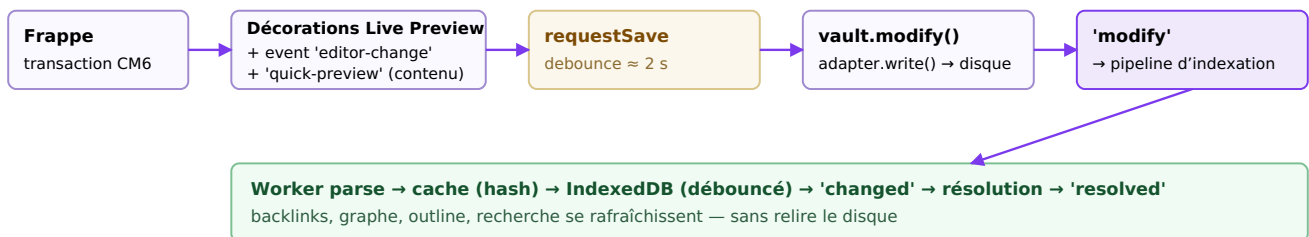


Fig. 3 — Flux d'édition. Noter le double chemin : l'UI réagit immédiatement (décorations, `quick-preview`) ; le disque et l'index suivent, débouncés.

La subtilité à retenir : les vues dérivées n'attendent pas la sauvegarde. `quick-preview` transporte le contenu *tapé* ; l'outline et la recherche se mettent à jour pendant le debounce. Si l'app crashe entre-temps, une note tapée depuis <2 s peut se perdre — compromis choisi, mitigé par le plugin interne « file recovery » qui snapshotte périodiquement INF.

16.3 Renommer un fichier sans rien casser

```

fileManager.renameFile(file, nouveauChemin)
  → inversion de resolvedLinks : qui pointe vers file ?
  → pour chaque source : réécriture chirurgicale des liens aux offsets du cache
    (Vault.process – atomique ; LinkUpdaters par format, dont .canvas)
  → vault.rename(file) – l'objet TFile est muté, identité conservée
  → événement 'rename'(file, oldPath) → fileCache re-clefé (le hash, lui, n'a pas changé
    → l'entrée metadataCache est réutilisée telle quelle : coût ≈ 0)
  → 'resolve' des fichiers touchés → 'resolved'
  
```

16.4 Activer un plugin à chaud

```
Réglages → toggle → plugins.enablePluginAndSave(id)
  → community-plugins.json mis à jour (debounce) → loadPlugin(id) : lecture de main.js,
    éval CJS (require shim), new PluginClass(app, manifest)
  → plugin.load() → onload() : le plugin remplit les registres (§3.2)
  → onUserEnable() (première activation seulement) ; 'layout-change'/updateOptions
    propagent les nouvelles extensions d'éditeur aux éditeurs ouverts
Désactivation : unload() → rejeu inverse de tous les register* → registres nettoyés
```

17. Bibliothèques externes : la liste exacte

Liste officielle et complète (page « Credits », consultée en juillet 2026) **CRED**, annotée du rôle inféré dans l'architecture :

Bibliothèque	Version	Licence	Rôle dans Obsidian
Electron	37.3.0	MIT	Coque desktop (Chromium + Node).
Capacitor	5.x	MIT	Coque mobile iOS/Android ; ponts natifs du <code>CapacitorAdapter</code> .
CodeMirror	6 (+Vim 6.0.0)	MIT	Moteur d'édition ; mode Vim optionnel.
Lezer	—	MIT	Parsing incrémental de l'éditeur (grammaire Markdown côté CM6).
remark	—	MIT	Lignée du parseur Markdown du mode lecture (fork maison probable ; le crédit cite aussi le copyright de <i>marked</i>).
DOMPurify	—	MPL 2.0	Assainissement du HTML rendu.
Prism	1.29.0	MIT	Coloration des blocs de code en lecture (chargé à la demande).
MathJax	—	Apache 2.0	Rendu LaTeX (chargé à la demande).
Mermaid	11.4.1	MIT	Diagrammes texte (chargé à la demande).
pdf.js	—	Apache 2.0	Visionneuse PDF (une View comme une autre, §8.3).
PixiJS	—	MIT	Rendu WebGL de la vue graphe ; exposé en global <code>PIXI</code> .
rbush	—	MIT	Index spatial R-tree (hit-testing/culling graphe et canvas, inféré).
Moment.js	2.29.4	MIT	Dates ; réexporté par l'API (<code>import { moment } from "obsidian"</code>) : dépendance gelée pour la stabilité de l'écosystème.
Lucide	0.446.0	ISC	Jeu d'icônes, derrière le registre d'icônes.
i18next	—	MIT	Localisation (<code>getLanguage()</code> public).
YAML	2.7.0	ISC	Frontmatter, Bases (<code>parseYaml/stringifyYaml</code> publics).
Turndown	—	MIT	HTML → Markdown (<code>htmlToMarkdown()</code> , collage, import).
reveal.js	4.3.1	MIT	Plugin interne « Slides ».
scrypt	—	Apache 2.0	Dérivation de clef du chiffrement E2E de Sync.
Webpack	—	MIT	Bundling de l'application.

Liste courte, versions épinglées, zéro framework UI, zéro ORM, zéro state manager : la surface de dépendances est une décision d'architecture en soi. Les modules réexportés aux plugins (`obsidian`, `moment`, `@codemirror/*`, `electron`) forment un contrat de compatibilité que l'équipe s'interdit de casser.

18. Feuille de route pour votre clone

18.1 Correspondance de stack proposée

Sous-système d'Obsidian	Choix recommandé pour un clone (2026)
Coque	Electron + TypeScript strict ; esbuild/Vite pour le dev. Prévoir l'équivalent <code>DataAdapter</code> dès le jour 1 même sans mobile.
Éditeur	CodeMirror 6 + <code>@codemirror/lang-markdown</code> (Lezer). Écrivez votre façade <code>Editor</code> immédiatement : n'exposez jamais CM6 nu à vos plugins.
Rendu lecture	<code>unified/remark+rehype</code> (ou <code>markdown-it</code>) + <code>DOMPurify</code> ; chaîne de post-processeurs ordonnée dès la v0.
Index	Web Worker (<code>comlink</code>) + <code>IndexedDB</code> (<code>idb</code>) ; reproduire le schéma <code>fileCache/metadataCache</code> adressé par hash — c'est peu de code pour beaucoup d'effet.
Watcher	<code>chokidar</code> (desktop) ; réconciliation <code>mtime/size</code> au boot.
Graphe	<code>PixiJS</code> (ou <code>sigma.js</code>) + <code>d3-force</code> dans un worker ; interpolation côté rendu ; veille sur trames stables.
Icônes / dates / YAML	<code>Lucide</code> ; <code>date-fns</code> ou <code>dayjs</code> (n'imitiez pas le gel de Moment — c'est un héritage, pas un modèle) ; <code>yaml</code> .

18.2 Ordre de construction conseillé

L'erreur classique serait de commencer par l'éditeur. La dépendance réelle des sous-systèmes suggère l'ordre inverse : **(1)** `Component` + `Events/EventRef` (deux cents lignes, tout en dépend) ; **(2)** `DataAdapter` + `Vault` + `watcher` + événements ; **(3)** l'index (worker, hash, `IndexedDB`, `resolvedLinks`) — à ce stade, backlinks et graphe sont presque gratuits ; **(4)** `Workspace` composite + sérialisation + `ViewRegistry` (avec vues différées d'emblée) ; **(5)** l'éditeur derrière sa façade, live preview minimal (masquage de marqueurs d'abord, widgets ensuite) ; **(6)** le rendu lecture + post-processeurs ; **(7)** le chargeur de plugins — et migrez immédiatement une de vos features cœur dessus pour le valider (§11.6) ; **(8)** graphe, recherche, palette.

18.3 Les douze patterns à emporter

Pattern	Où Obsidian l'applique
1. Cycle de vie hiérarchique à nettoyage déclaré (<code>Component</code>)	Tout objet vivant ; les <code>register*</code> des plugins (§4).
2. Pub/sub à références désinscriptibles (<code>EventRef</code>)	<code>Vault</code> , cache, workspace ; s'emboîte dans le n°1 (§5).
3. Registres mutables comme points d'extension	Vues, embeds, commandes, icônes, protocoles, widgets (§3.2).
4. Bridge stockage (<code>DataAdapter</code>)	Desktop/mobile/(web) ; le cœur ignore le FS (§6.1).
5. Couche sémantique séparée du stockage	<code>FileManager</code> vs <code>Vault</code> (§6.5).
6. Index hors-thread, adressé par contenu, persistant mais jetable	<code>MetadataCache</code> : worker + hash + <code>IndexedDB</code> (§7.2).
7. Le graphe comme projection de l'index	<code>resolvedLinks</code> → backlinks, graphe, liens morts (§7.3).
8. Composite sérialisable + état éphémère séparé	<code>Workspace</code> , <code>ViewState</code> vs <code>ephemeralState</code> (§8.2).
9. Paresse structurelle	<code>DeferredView</code> , <code>loadPrism/MathJax/Mermaid</code> , popouts (§8.4, §10.3).

Pattern	Où Obsidian l'applique
10. Façade anti-corruption sur la dépendance volatile	Editor sur CM5→CM6 ; StateFields ponts (§9).
11. Débouncing/coalescence à chaque frontière	requestSave*, didFinish, transactionSave (§6.3, §7.5).
12. Dogfooding du système de plugins	~30 features cœur = plugins internes (§11.6).

18.4 Pièges identifiés en chemin

Quelques difficultés que l'architecture d'Obsidian révèle en creux, à anticiper : la **normalisation des chemins** (Unicode NFC/NFD entre macOS et le reste, sensibilité à la casse — d'où `normalizePath()` public) ; la **cohérence des deux parseurs** Markdown (chaque divergence devient un bug visible en basculant de mode) ; les **écritures atomiques** (écrire-temporaire-puis-renommer, sous peine de fichiers tronqués en cas de crash — le contrat `process()` n'a de valeur que si l'adapter le garantit) ; le **realm unique multi-fenêtres** (§2.4) qui interdit les captures naïves de `document` ; et la **politique de debounce** qui doit être explicite par frontière, faute de quoi les plugins s'étalonnent sur des comportements accidentels.

19. Annexe : hiérarchie de classes consolidée

Vue d'ensemble des hiérarchies principales de l'API publique API (sélection ; les familles Bases et Setting* sont résumées) :

```

Component
├─ Plugin                                (abstrait ; classe mère des plugins)
├─ View (abstrait)
│   └─ ItemView
│       ├── FileView
│       │   └─ EditableFileView
│       │       └─ TextView
│       │           └─ MarkdownView    (editor + previewMode)
│       └─ (vues sans fichier : recherche, outline, graphe...)
├─ MarkdownRenderChild
│   └─ MarkdownRenderer
│       └─ MarkdownPreviewView        (implémente MarkdownSubView)
├─ HoverPopover
├─ Menu                                (implémente HistoryHandler)
└─ BasesView / QueryController        (sous-système Bases)

Events
├─ Vault                                (adapter : DataAdapter)
├─ MetadataCache
├─ Workspace
├─ WorkspaceItem
│   └─ WorkspaceParent
│       ├── WorkspaceSplit
│       │   ├── WorkspaceContainer
│       │   │   ├── WorkspaceRoot
│       │   │   └─ WorkspaceWindow    (popout)
│       │   └─ WorkspaceSidedock
│       ├── WorkspaceTabs
│       ├── WorkspaceFloating
│       └─ WorkspaceMobileDrawer
│   (feuille : WorkspaceLeaf – héberge une View)
├─ SecretStorage
└─ (internes : Plugins, InternalPlugins, Commands, ViewRegistry, EmbedRegistry...)

BaseComponent
├─ ButtonComponent / ExtraButtonComponent / SecretComponent
└─ ValueComponent<T>
    ├── AbstractTextComponent → TextComponent / TextAreaComponent / SearchComponent
    ├── ToggleComponent · SliderComponent · DropdownComponent
    └─ ColorComponent · ProgressBarComponent · MomentFormatComponent

Modal (implémente HistoryHandler)
├─ ConfirmationModal
└─ SuggestModal<T>
    └─ FuzzySuggestModal<T>

PopoverSuggest<T> (implémente ISuggestOwner, HistoryHandler)
├─ EditorSuggest<T>
└─ AbstractInputSuggest<T>

TAbstractFile — TFile (stat, basename, extension) · TFolder (children)
Editor (abstrait – façade CM6) ; Scope / Keymap ; Notice ; Tasks ; Debouncer
Value → NotNullValue → PrimitiveValue<T> → String/Number/Boolean/Date/List/Object... (Bases)

```

Sources

- Typages officiels de l'API : paquet npm `obsidian` (`obsidian.d.ts`, v1.12.x) — github.com/obsidianmd/obsidian-api (et son CHANGELOG).
- Documentation développeur officielle — docs.obsidian.md, notamment le guide « Defer views » ([/plugins/guides/defer-views](https://docs.obsidian.md/plugins/guides/defer-views)).
- Crédits officiels et bibliothèques tierces — help.obsidian.md/credits.
- « How Obsidian stores data » (cache IndexedDB) — [help Obsidian](https://help.obsidian.md).
- Typages communautaires des API internes — github.com/Fevol/obsidian-typings (analysés ici en version 1.12.7 via npm).
- Analyse communautaire du DOM Live Preview — forum.obsidian.md ; guide de migration Live Preview du hub communautaire.
- Format Canvas ouvert — jsoncanvas.org ; plugin d'exemple officiel — [obsidian-sample-plugin](https://obsidian-sample-plugin.github.io).

Document généré le 6 juillet 2026. Les éléments marqués COMM/INF décrivent des mécanismes internes non contractuels, susceptibles de changer à toute version. Obsidian est une marque d'Obsidian.md / Dynalist Inc. ; ce document n'est ni affilié ni approuvé par l'éditeur.